在本章中，我们将讨论创建和微调嵌入模型以提高其代表性和语义能力的各种方法。

一、嵌入模型（Embedding Models）

什么是嵌入？非结构化文本数据本身通常很难处理。它们不是我们可以直接处理、可视化和从中创建可操作结果的。首先，我们必须将这些文本数据转换为易于处理的数据：数字表示。这一过程通常被称为将输入嵌入到输出可用向量中，即嵌入。

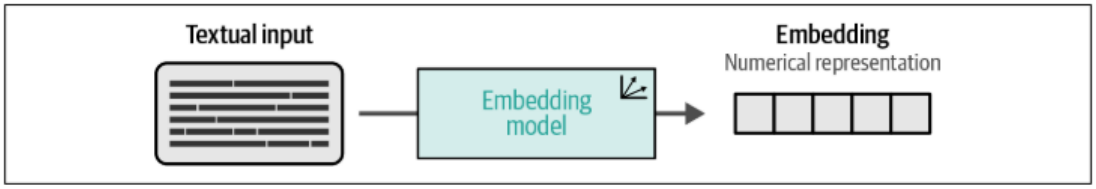


Figure 10-1. We use an embedding model to convert textual input, such as documents, sentences, and phrases, to numerical representations, called embeddings.

这个嵌入输入的过程通常由 LLM 执行，我们将其称为嵌入模型。这种模型的主要目的是尽可能准确地将文本数据表示为嵌入。

什么是代表性的准确呢？通常情况下，我们希望捕获文档的语义本质--含义。在实践中，我们期望彼此相似的文档向量是相似的，而每个讨论完全不同的东西的文档的嵌入应该是不相似的。相似的向量在嵌入空间中距离小，不相似的向量在嵌入空间中距离远。此图为简化示例。虽然二维可视化有助于说明嵌入的接近性和相似性，但这些嵌入通常驻留在高维空间中。

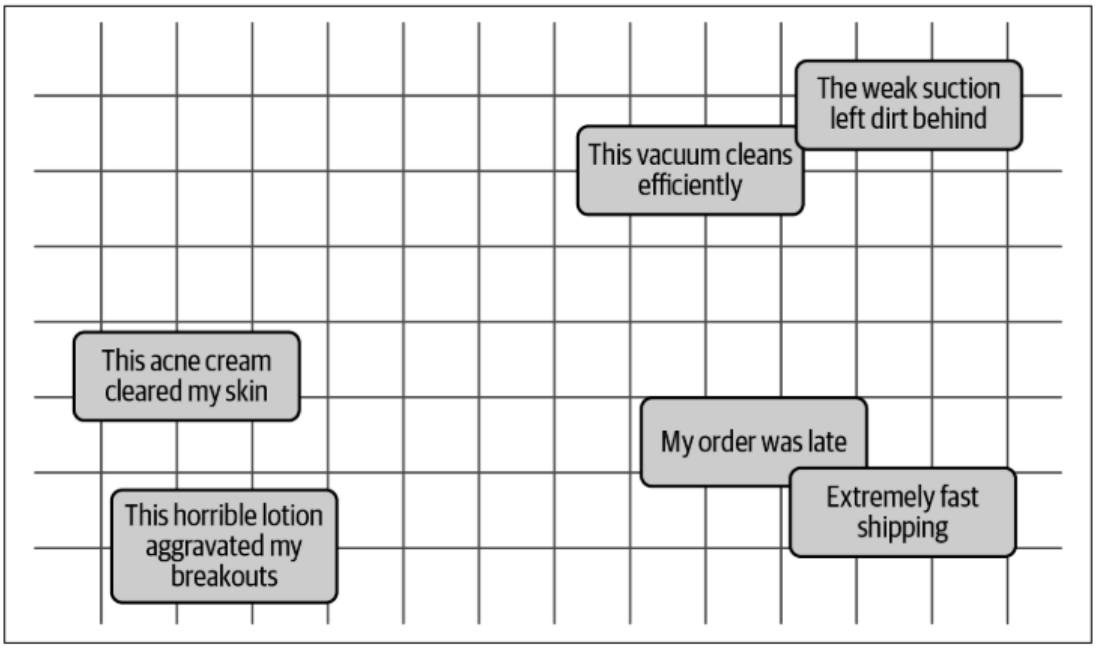


Figure 10-2. The idea of semantic similarity is that we expect textual data with similar meanings to be closer to each other in n-dimensional space (two dimensions are illustrated here).

二、对比学习（Contrastive Learning）

用于训练和微调文本嵌入模型的一种主要技术称为对比学习。对比学习是一种旨在训练嵌入模型的技术，使得相似的文档在向量空间中更接近，而不相似的文档则更远离。

对比学习的基本思想是，学习和建模文档之间相似性/不相似性的最佳方法是提供相似和不相似对的示例，以便模型了解是什么使其不同或相似。

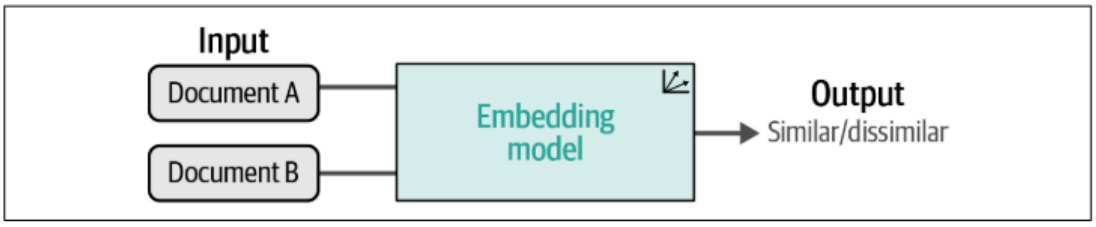


Figure 10-4. Contrastive learning aims to teach an embedding model whether documents are similar or dissimilar. It does so by presenting groups of documents to a model that are similar or dissimilar to a certain degree.

另一种看待对比学习的方法是通过解释的本质。例如，为什么这张图是一只猫而不是一只狗。这被称为对比解释，为什么是 P 而不是 Q。通过提供两个概念之间的对比，它开始学习定义概念的特征，以及不相关的特征。当我们把一个问题作为对比时，我们会得到更多的信息。

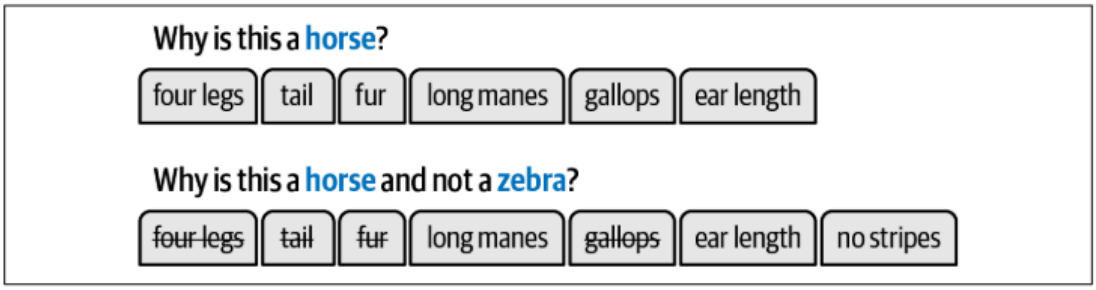


Figure 10-5. When we feed an embedding model different contrasts (degrees of similarity), it starts to learn what makes things different from one another and thereby the distinctive characteristics of concepts.

三、SBERT（Sentence-transformer）

虽然有许多形式的对比学习，但在自然语言处理社区中推广该技术的一个框架是BERCE-transformers。它的方法解决了原始 BERT 实现用于创建句子嵌入的主要问题，即其计算效率的问题，填补了原始 BERT 在句向量表示上的缺陷。

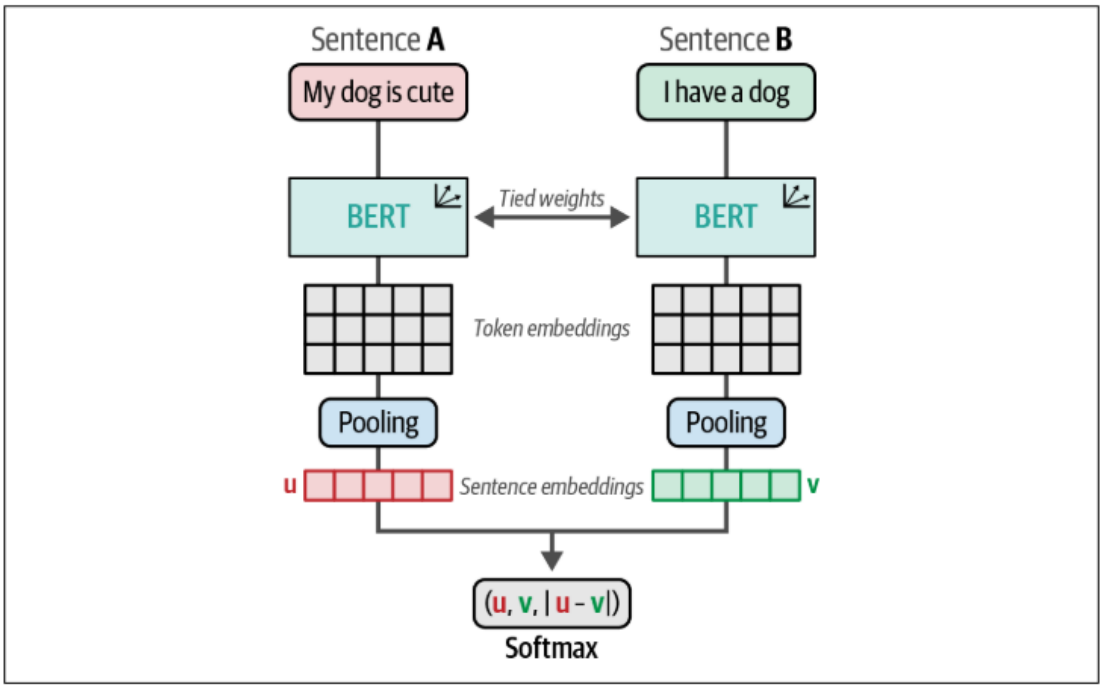


Figure 10-7. The architecture of the original `sentence-transformers` model, which leverages a Siamese network, also called a bi-encoder.

Sentence-transformer 训练使用了一种连体结构。如图 10-7 所示，我们有两个相同的 BERT 模型，它们共享相同的权重和神经架构。这些模型被馈送句子，从这些句子通过标记嵌入的池生成嵌入。在此基础上，利用句子嵌入的相似度对模型进行优化。由于两个 BERT 模型的权重是相同的，因此我们可以使用单个模型，并将句子一个接一个地馈送给它。

这些句子对的优化过程是通过损失函数完成的，这可能会对模型的性能产生重大影响。在训练过程中，每个句子的嵌入与嵌入之间的差异连接在一起。然后，通过 softmax 分类器优化此结果嵌入。

接下来展示 SBERT 的训练过程。为了进行对比学习，我们需要两件事。首先，我们需要构成相似/不相似对的数据。其次，我们需要定义模型如何定义和优化相似性。

在预训练嵌入模型时，经常会看到使用来自自然语言推理（NLI）数据集的数据。NLI 指的是调查对于给定的前提，它是否需要假设（蕴涵 entailment），相矛盾（矛盾 Contradiction）或两者都不（中性 neutral）的任务。

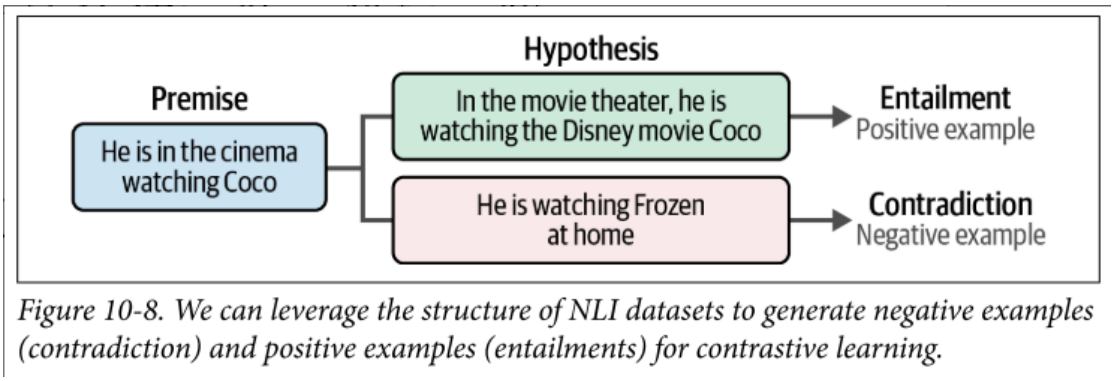
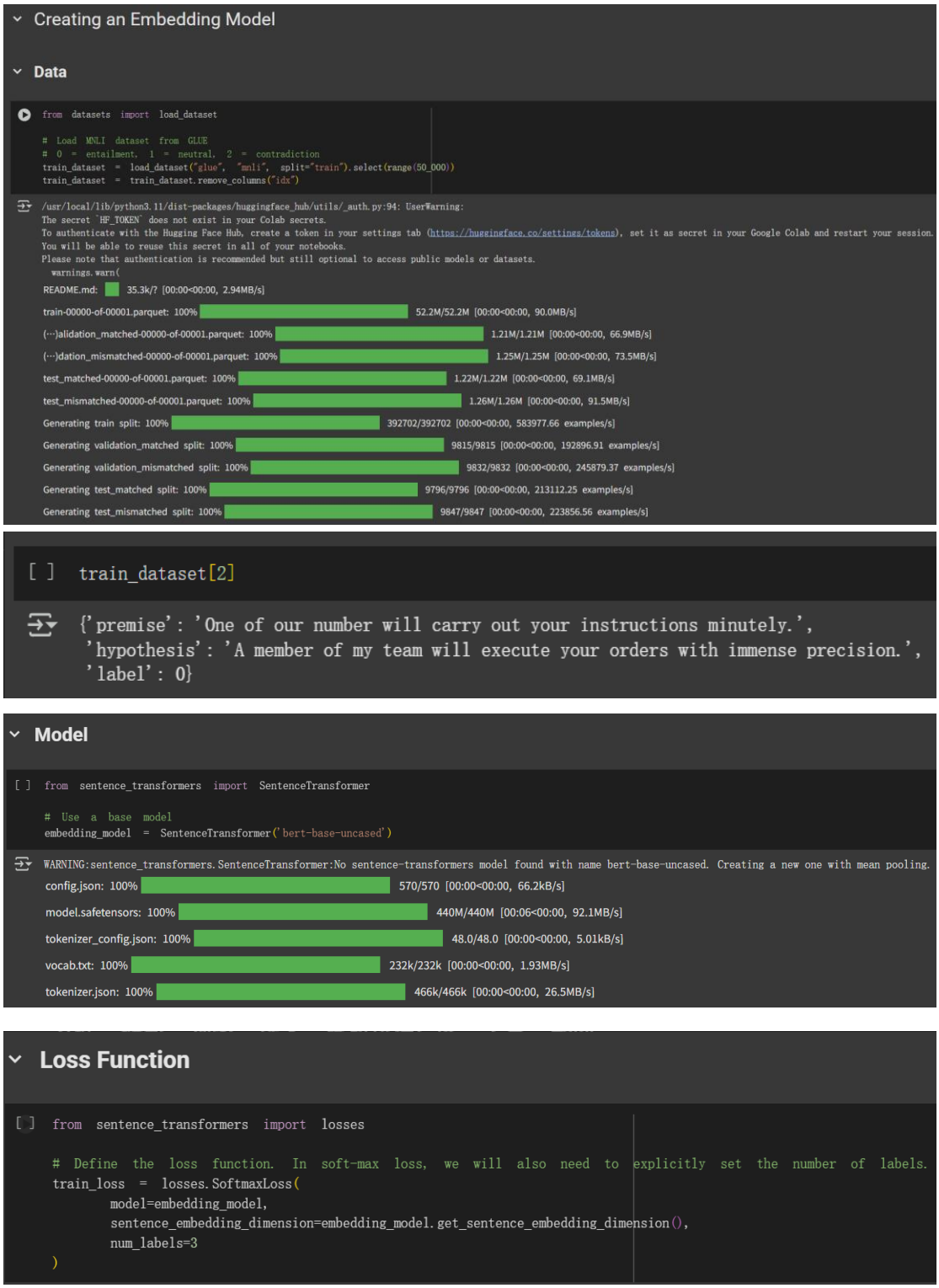


Figure 10-8. We can leverage the structure of NLI datasets to generate negative examples (contradiction) and positive examples (entailments) for contrastive learning.

例如，当前提是“他在电影院看寻梦环游记”，假设“他在家里看冰雪奇缘”，那么这些陈述是矛盾的 Contradiction。相反，当前提是“他在电影院里看寻梦环游记”，假设是“他在电影院里看迪士尼的电影寻梦环游记”时，这些陈述被认为是蕴涵 entailment。

使用的数据来自 General Language Understanding Evaluation benchmark (GLUE)。这个 GLUE 基准测试包括九个语言理解任务，用于评估和分析模型性能。其中一个任务是 Multi-Genre Natural Language Inference（MNLI）语料库，它是 392,702 个用 contradiction，neutral，entailment 注释的句子对的集合。我们将使用数据的一个子集，50，000 个带注释的句子对，来创建一个训练时间不超过几个小时的最小示例。

四、创建嵌入模型



我们可以使用 Semantic Textual Similarity Benchmark（STSB）来评估我们模型的性能。

Evaluation

```
[ ] from sentence_transformers.evaluation import EmbeddingSimilarityEvaluator

    # Create an embedding similarity evaluator for stsb
    #构建了一个评估器来计算模型预测的句子嵌入之间的相似度（如余弦相似度）与人工标注的真实相似度之间的相关性。
    val_sts = load_dataset('glue', 'stsb', split='validation')
    evaluator = EmbeddingSimilarityEvaluator(
            sentences1=val_sts["sentence1"],
            sentences2=val_sts["sentence2"],
            scores=[score/5 for score in val_sts["label"]],
            main_similarity="cosine",
    )
```

```
train-00000-of-00001.parquet: 100% 502k/502k [00:00<00:00, 14.1MB/s]
validation-00000-of-00001.parquet: 100% 151k/151k [00:00<00:00, 10.3MB/s]
test-00000-of-00001.parquet: 100% 114k/114k [00:00<00:00, 12.8MB/s]
Generating train split: 100% 5749/5749 [00:00<00:00, 183899.25 examples/s]
Generating validation split: 100% 1500/1500 [00:00<00:00, 71619.97 examples/s]
Generating test split: 100% 1379/1379 [00:00<00:00, 87307.47 examples/s]
```

Training

```
[ ] from sentence_transformers.training_args import SentenceTransformerTrainingArguments

    # Define the training arguments
    args = SentenceTransformerTrainingArguments(
            output_dir="base_embedding_model",
            num_train_epochs=1,
            per_device_train_batch_size=32,
            per_device_eval_batch_size=32,
            warmup_steps=100,
            fp16=True,
            eval_steps=100,
            logging_steps=100,
    )
```

```
from sentence_transformers.trainer import SentenceTransformerTrainer

# Train embedding model
trainer = SentenceTransformerTrainer(
        model=embedding_model,
        args=args,
        train_dataset=train_dataset,
        loss=train_loss,
        evaluator=evaluator
)
trainer.train()
```

[1563/1563 06:10, Epoch 1/1]

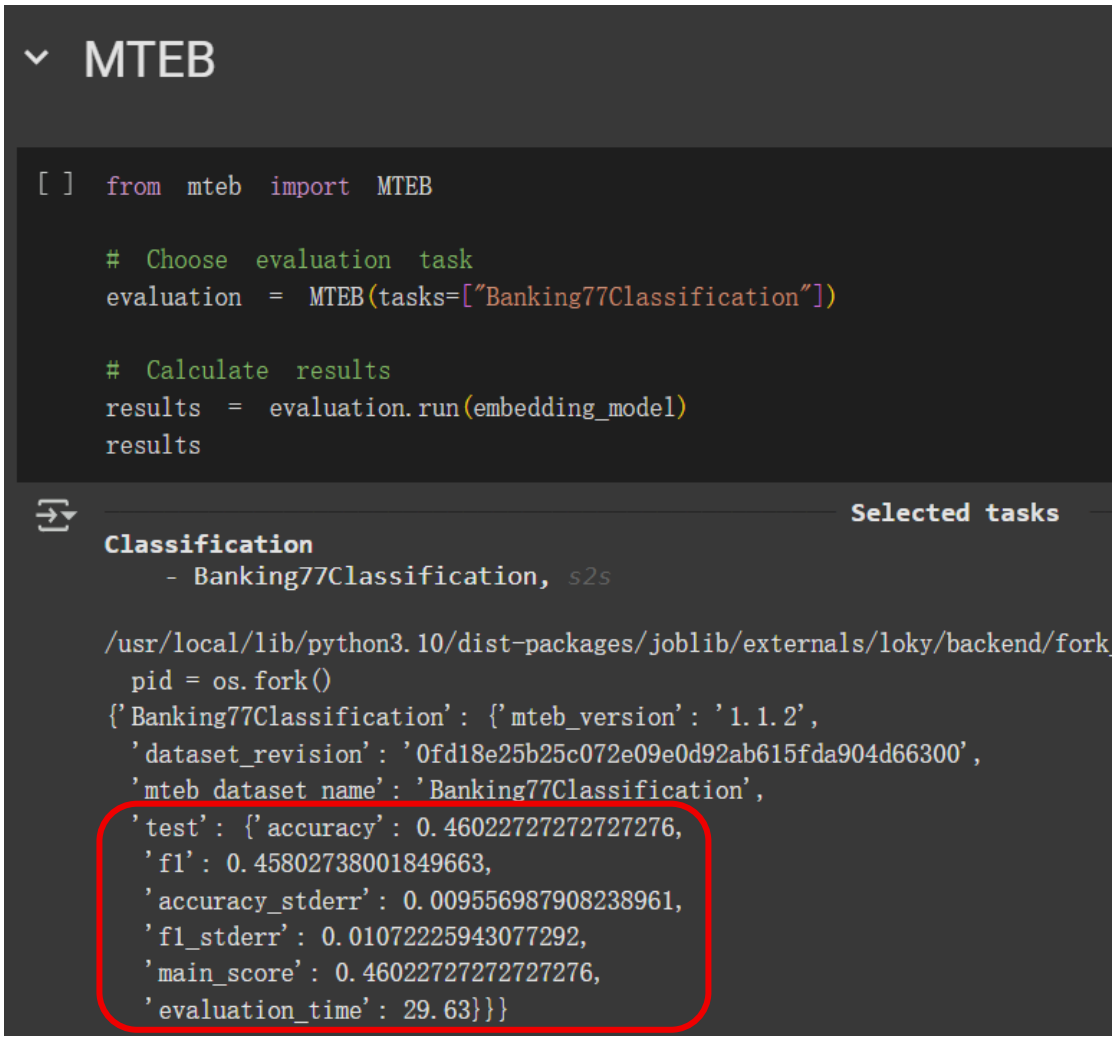
Step	Training Loss
100	1.080700
200	0.959400
300	0.916200
400	0.870200
500	0.849100
600	0.854200
700	0.835200
800	0.825200
900	0.818100
1000	0.800300
1100	0.781600
1200	0.777100
1300	0.786600
1400	0.767900
1500	0.797100

```
[ ]  # Evaluate our trained model
     evaluator(embedding_model)
```

```
{'pearson_cosine': 0.3710938716460552,
 'spearman_cosine': 0.45148122260403883,
 'pearson_manhattan': 0.4037396904694362,
 'spearman_manhattan': 0.4396893995197567,
 'pearson_euclidean': 0.390788259199341,
 'spearman_euclidean': 0.43444104358464286,
 'pearson_dot': 0.3392927926047231,
 'spearman_dot': 0.3530708415227247,
 'pearson_max': 0.4037396904694362,
 'spearman_max': 0.45148122260403883}
```

一个好的嵌入模型不仅仅是在 STSB 基准测试（评估器使用的是 STSB）中取得好成绩。正如我们之前所观察到的，GLUE 基准测试有许多任务可以评估我们的嵌入模型。然而，还有更多的基准允许嵌入模型的评估。为了统一这一评估程序，开发了海量文本嵌入基准 Massive Text Embedding Benchmark（MTEB）。5 MTEB 涵盖 8 个嵌入任务，涵盖 58 个数据集和 112 种语言。

这段代码使用 MTEB 库来评估句子嵌入模型 embedding_model 在特定任务 Banking77Classification 上的性能，该任务包含在线银行场景的 77 个意图类别（如"转账失败"、"查询余额"等）。



这为我们提供了这个特定任务的几个评估指标，我们可以使用它们来探索其性能。

对于损失函数 loss function，有两个经常被使用，余弦相似性 Cosine similarity 和多负例排序损失 Multiple negatives ranking (MNR) loss。

①余弦相似性损失是一种直观且易于使用的损失，它通常用于语义文本相似性任务。在这些任务中，模型会给出数据集中文本对的相似性得分，该值介于 0

与 1 之间，0 表示相异，1 表示相同。

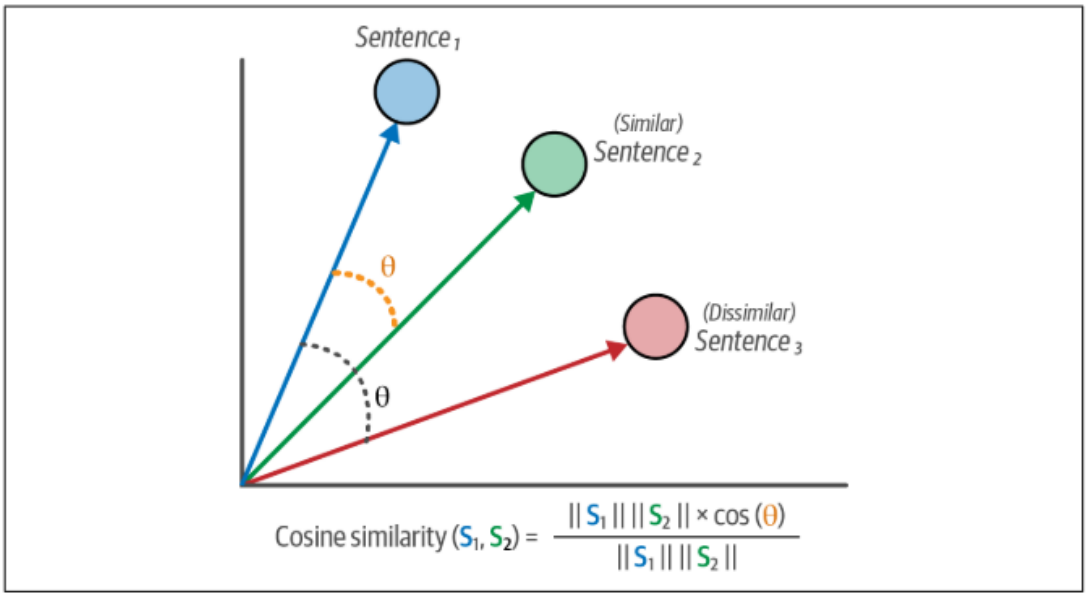


Figure 10-9. Cosine similarity loss aims to minimize the cosine distance between semantically similar sentences and to maximize the distance between semantically dissimilar sentences.

Cosine Similarity Loss

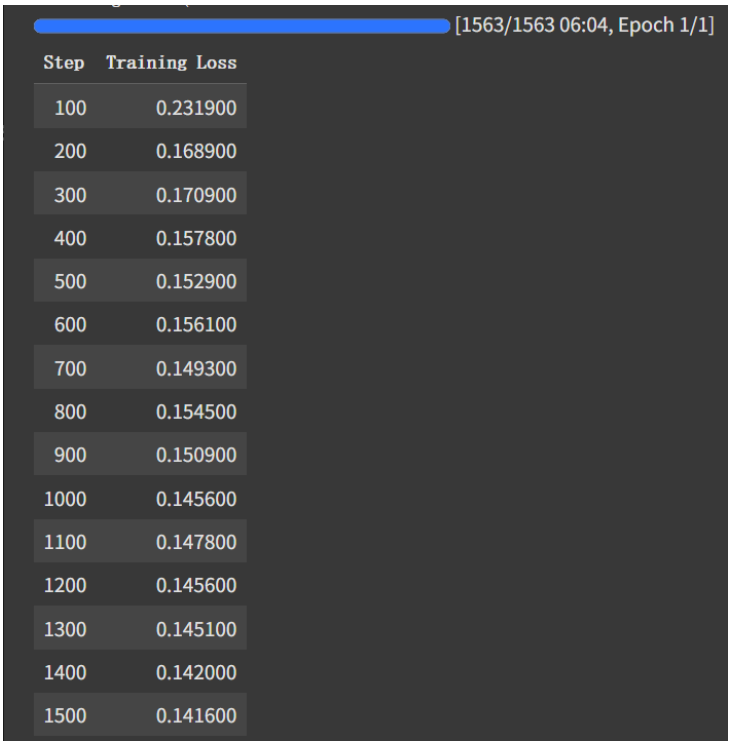
```
[ ]  from datasets import Dataset, load_dataset

     # Load MNLI dataset from GLUE
     # 0 = entailment, 1 = neutral, 2 = contradiction
     train_dataset = load_dataset("glue", "mnli", split="train").select(range(50_000))
     train_dataset = train_dataset.remove_columns("idx")

     # (neutral/contradiction)=0 and (entailment)=1
     mapping = {2: 0, 1: 0, 0:1}
     train_dataset = Dataset.from_dict({
            "sentence1": train_dataset["premise"],
            "sentence2": train_dataset["hypothesis"],
            "label": [float(mapping[label]) for label in train_dataset["label"]]
     })
```

余弦相似性损失计算两个文本的两个嵌入之间的余弦相似性，并将其与标记的相似性分数进行比较。余弦相似性损失直观地适用于具有表示 0 和 1 之间相似性的成对句子和标签的数据。为了在 NLI 数据集上使用这种损失，我们需要将蕴涵（0）、中性（1）和矛盾（2）标签转换为 0 和 1。entailment 蕴涵代表了高度的相似性，赋值 1，neutral 和 contradiction 代表了相异性，赋值 0.

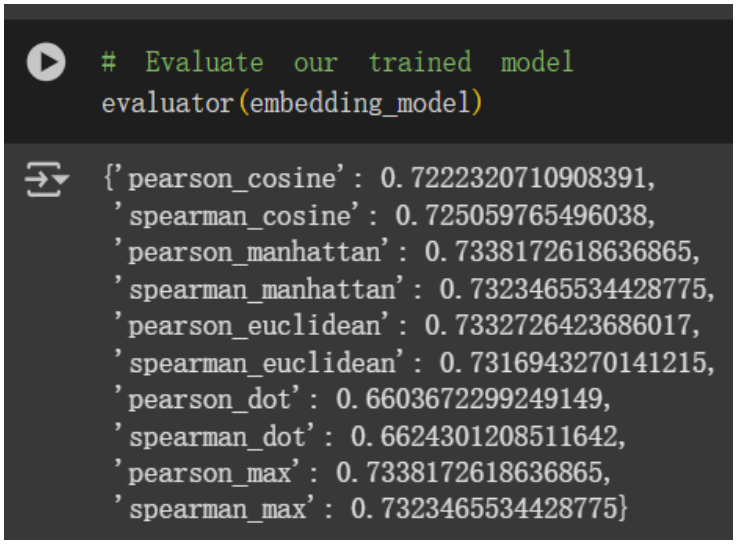
接下来的步骤与之前一样，创建评估器和训练模型。

```
[ ]  from sentence_transformers.evaluation import EmbeddingSimilarityEvaluator

     # Create an embedding similarity evaluator for stsb
     val_sts = load_dataset('glue', 'stsb', split='validation')
     evaluator = EmbeddingSimilarityEvaluator(
             sentences1=val_sts["sentence1"],
             sentences2=val_sts["sentence2"],
             scores=[score/5 for score in val_sts["label"]],
             main_similarity="cosine"
     )
```

```
     from sentence_transformers import losses, SentenceTransformer
     from sentence_transformers.trainer import SentenceTransformerTrainer
     from sentence_transformers.training_args import SentenceTransformerTrainingArguments

     # Define model
     embedding_model = SentenceTransformer('bert-base-uncased')

     # Loss function
     train_loss = losses.CosineSimilarityLoss(model=embedding_model)

     # Define the training arguments
     args = SentenceTransformerTrainingArguments(
             output_dir="cosineloss_embedding_model",
             num_train_epochs=1,
             per_device_train_batch_size=32,
             per_device_eval_batch_size=32,
             warmup_steps=100,
             fp16=True,
             eval_steps=100,
             logging_steps=100,
     )

     # Train model
     trainer = SentenceTransformerTrainer(
             model=embedding_model,
             args=args,
             train_dataset=train_dataset,
             loss=train_loss,
             evaluator=evaluator
     )
     trainer.train()
```



在训练后评估模型，我们得到以下分数：与 softmax 损失示例（得分为 0.37）相比，Pearson 余弦得分为 0.72 是一个很大的改进。



②Multipple negatives ranking loss 多负例排序损失

Multiple Negatives Ranking (MNR) loss（多负例排序损失）是一种高效的对比学习损失函数，其核心思想是通过单批次内的多负例对比，使正样本在嵌入空间中靠近查询样本，同时远离多个负样本。它不依赖外部负例采样，而是直接利用批次内其他样本作为负例，例如批次大小为 N，则每个查询隐式可获得 1 个正例，N−1 个负例进行训练，这 N 个样本都可以用来训练，效率高。

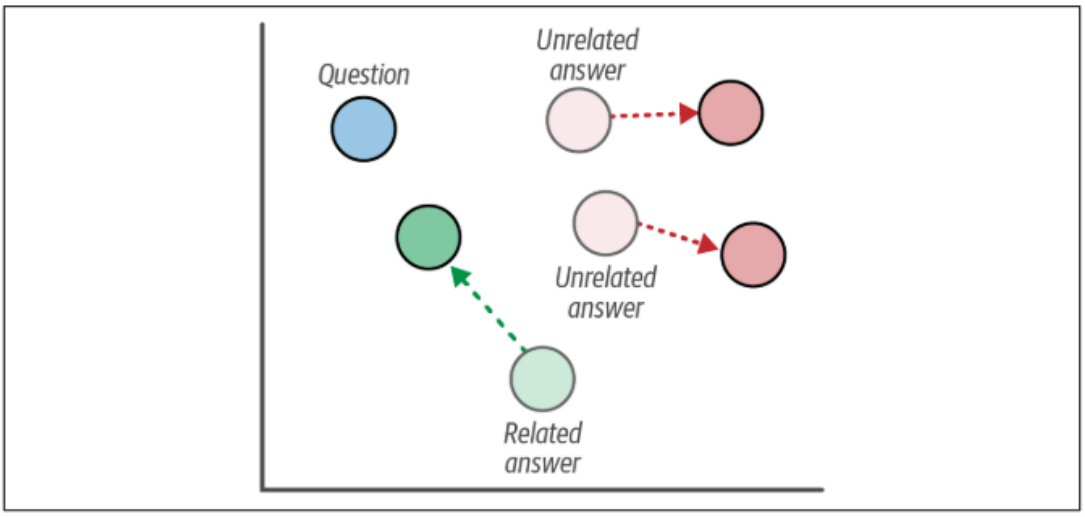


Figure 10-10. Multiple negatives ranking loss aims to minimize the distance between related pairs of text, such as questions and answers, and maximize the distance between unrelated pairs, such as questions and unrelated answers.

在 MNLI 数据集中，我们首先将所有标签为 entailment 的样本选出来。然后将 premise 的句子作为 anchor，hypothesis 的句子作为 positive，那么 anchor 和 hypothesis 的句子对是相似的，作为正例。随后将数据集中的每个样本的 anchor 和其余样本的 hypothesis 相配对，由于两个句子不相似，所以可以作为负例用来训练。

Multiple Negatives Ranking Loss

```
[ ]  import random
     from tqdm import tqdm
     from datasets import Dataset, load_dataset

     # # Load MNLI dataset from GLUE
     mnli = load_dataset("glue", "mnli", split="train").select(range(50_000))
     mnli = mnli.remove_columns("idx")
     mnli = mnli.filter(lambda x: True if x['label'] == 0 else False)

     # Prepare data and add a soft negative
     train_dataset = {"anchor": [], "positive": [], "negative": []}
     soft_negatives = mnli["hypothesis"]
     random.shuffle(soft_negatives)
     for row, soft_negative in tqdm(zip(mnli, soft_negatives)):
             train_dataset["anchor"].append(row["premise"])
             train_dataset["positive"].append(row["hypothesis"])
             train_dataset["negative"].append(soft_negative)
     train_dataset = Dataset.from_dict(train_dataset)
     len(train_dataset)
```

```
16875it [00:01, 14110.96it/s]
16875
```

接下来的步骤与之前一样，创建评估器和训练模型。

```
[ ]  from sentence_transformers.evaluation import EmbeddingSimilarityEvaluator

     # Create an embedding similarity evaluator for stsb
     val_sts = load_dataset('glue', 'stsb', split='validation')
     evaluator = EmbeddingSimilarityEvaluator(
             sentences1=val_sts["sentence1"],
             sentences2=val_sts["sentence2"],
             scores=[score/5 for score in val_sts["label"]],
             main_similarity="cosine"
     )
```

```
     from sentence_transformers import losses, SentenceTransformer
     from sentence_transformers.trainer import SentenceTransformerTrainer
     from sentence_transformers.training_args import SentenceTransformerTrainingArguments

     # Define model
     embedding_model = SentenceTransformer('bert-base-uncased')

     # Loss function
     train_loss = losses.MultipleNegativesRankingLoss(model=embedding_model)

     # Define the training arguments
     args = SentenceTransformerTrainingArguments(
             output_dir="mnrloss_embedding_model",
             num_train_epochs=1,
             per_device_train_batch_size=32,
             per_device_eval_batch_size=32,
             warmup_steps=100,
             fp16=True,
             eval_steps=100,
             logging_steps=100,
     )

     # Train model
     trainer = SentenceTransformerTrainer(
             model=embedding_model,
             args=args,
             train_dataset=train_dataset,
             loss=train_loss,
             evaluator=evaluator
     )
     trainer.train()
```

[528/528 03:00, Epoch 1/1]

Step	Training Loss
100	0.345200
200	0.105500
300	0.079000
400	0.062200
500	0.069000

```
# Evaluate our trained model
evaluator(embedding_model)
```

```
{'pearson_cosine': 0.8070727434643791,
 'spearman_cosine': 0.8106193672462586,
 'pearson_manhattan': 0.8213132116968124,
 'spearman_manhattan': 0.8164551132664518,
 'pearson_euclidean': 0.820988086354926,
 'spearman_euclidean': 0.8160139830687847,
 'pearson_dot': 0.7429357515240518,
 'spearman_dot': 0.7316164586329814,
 'pearson_max': 0.8213132116968124,
 'spearman_max': 0.8164551132664518}
```

负例的质量也有高低之分。由于负例是从其他问题/答案对中采样的，因此我们使用的这些负例答案可能与问题完全无关。因此，嵌入模型找到问题的正确答案变得非常容易。相反，我们希望得到与问题非常相关的否定，但不是正确的答案。这种负例被称为 hard negative。由于这将使嵌入模型的任务更加困难，因为它必须学习更多细微差别的表示，嵌入模型的性能通常会提高很多。

例子：问题如下：“阿姆斯特丹有多少人？”这个问题的一个相关答案是：“几乎有一百万人住在阿姆斯特丹。为了生成一个好的 hard negative，我们理想情况下希望答案包含关于阿姆斯特丹和居住在这个城市的人数的信息。例如：“乌得勒支有一百多万人，比阿姆斯特丹还多。”这个答案与问题有关，但并不是一个确切的答案，所以这是一个很好的 hard negative。

下图说明了 easy negative、semi-hard negative 和 hard negative 的区别。Easy negative 通常与问题和答案都无关。Semi-negative 与问题和答案的主题有一些相似之处，但有些不相关。Hard negative 与问题非常相似，但通常是错误的答案。

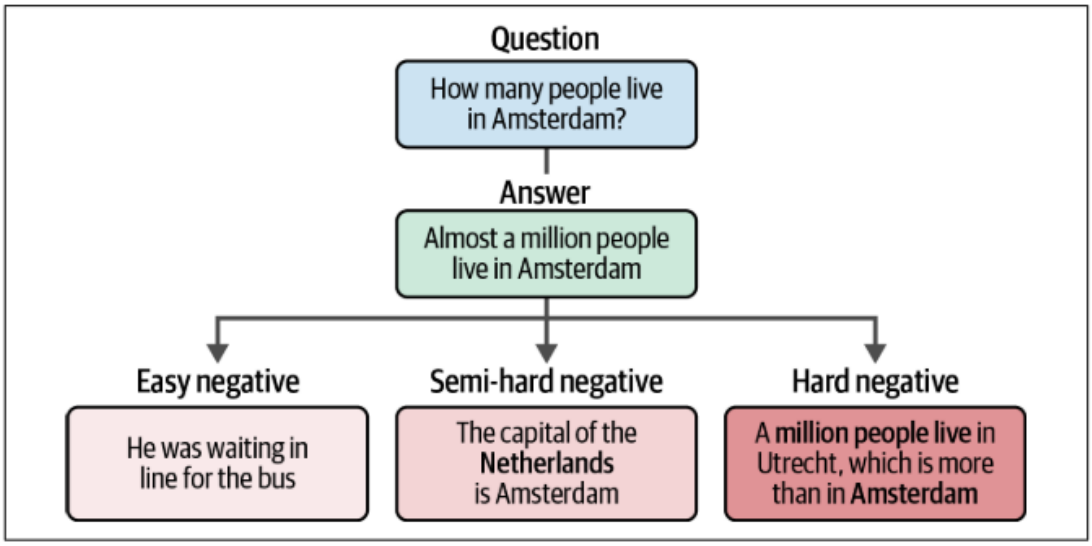


Figure 10-11. An easy negative is typically unrelated to both the question and answer. A semi-hard negative has some similarities to the topic of the question and answer but is somewhat unrelated. A hard negative is very similar to the question but is generally the wrong answer.

五、微调嵌入模型

在上一节中，我们从头开始学习了训练嵌入模型的基础知识，并了解了如何利用损失函数来进一步优化其性能。这种方法虽然非常强大，但需要从头开始创建嵌入模型（其实是选取了预训练好的 bert-base-uncased 通用模型进行训练）。这个过程可能非常昂贵和耗时。

相反，sentence-transformers 框架允许几乎所有的嵌入式模型被用作微调的基础。我们可以选择一个已经在大量数据上训练过的嵌入模型，并根据我们的特定数据或目的对其进行微调。根据数据可用性和域，有几种方法可以微调模型。我们将介绍两种方法，并展示利用预训练嵌入模型的优势。SBERT 是一种模型架构和方法论。Sentence-transformer 是一个开源库。

2. sentence-transformers

- **本质**：一个**Python开源库**
- **诞生**：由SBERT原作者创建（GitHub仓库）
- **核心功能**：
  - 实现了SBERT架构
  - 提供**100+预训练嵌入模型**（包括SBERT变体）
  - 训练框架（支持自定义数据集）
  - 评估工具（如EmbeddingSimilarityEvaluator）

1. Supervised

微调嵌入模型最直接的方法是重复之前训练模型的过程，但我们可以用 sentence-transformers 库替换掉基础模型“bert-base-uncased”。有很多模型可供选择，all-MiniLM-L6-v2 在许多用例中表现良好，并且由于其体积小，速度相当快，因此选用该模型。使用与 MNR 损失示例中相同的数据集，但使用 all-MiniLM-L6-v2 嵌入模型进行微调。

Supervised

```
from datasets import load_dataset
from sentence_transformers.evaluation import EmbeddingSimilarityEvaluator

# Load MNLI dataset from GLUE
# 0 = entailment, 1 = neutral, 2 = contradiction
train_dataset = load_dataset("glue", "mnli", split="train").select(range(50_000))
train_dataset = train_dataset.remove_columns("idx")

# 定义一个评估器来评估模型在训练过程中的性能，这也决定了要保存的最佳模型
# Create an embedding similarity evaluator for stsb
val_sts = load_dataset('glue', 'stsb', split='validation')
evaluator = EmbeddingSimilarityEvaluator(
        sentences1=val_sts["sentence1"],
        sentences2=val_sts["sentence2"],
        scores=[score/5 for score in val_sts["label"]],
        main_similarity="cosine"
)
```

```python
from sentence_transformers import losses, SentenceTransformer
from sentence_transformers.trainer import SentenceTransformerTrainer
from sentence_transformers.training_args import SentenceTransformerTrainingArguments

# Define model
embedding_model = SentenceTransformer('sentence-transformers/all-MiniLM-L6-v2')

# Loss function
train_loss = losses.MultipleNegativesRankingLoss(model=embedding_model)

# Define the training arguments
args = SentenceTransformerTrainingArguments(
        output_dir="finetuned_embedding_model",
        num_train_epochs=1,
        per_device_train_batch_size=32,
        per_device_eval_batch_size=32,
        warmup_steps=100,
        fp16=True,
        eval_steps=100,
        logging_steps=100,
)

# Train model
trainer = SentenceTransformerTrainer(
        model=embedding_model,
        args=args,
        train_dataset=train_dataset,
        loss=train_loss,
        evaluator=evaluator
)
```

[1563/1563 01:41, Epoch 1/1]

Step	Training Loss
100	0.157300
200	0.110500
300	0.119900
400	0.118800
500	0.108300
600	0.101100
700	0.119600
800	0.098700
900	0.104100
1000	0.105200
1100	0.093700
1200	0.105600
1300	0.101400
1400	0.104800
1500	0.104700

```
TrainOutput(global_step=1563, training_loss=0.10938188622414265, metrics={'train_runt
0.10938188622414265, 'epoch': 1.0})
```

```
# Evaluate our trained model
evaluator(embedding_model)
```

```
{'pearson_cosine': 0.8495095266556748, 'spearman_cosine': 0.8488949666143948}
```

```
# Evaluate the pre-trained model
original_model = SentenceTransformer('sentence-transformers/all-MiniLM-L6-v2')
evaluator(original_model)
```

```
{'pearson_cosine': 0.8696194530233021, 'spearman_cosine': 0.8671631197908374}
```

Augmented SBERT

训练或微调这些嵌入模型的缺点是它们通常需要大量的训练数据。在这些模型中，许多都是用超过 10 亿个句子对训练的。通常情况下不可能提取如此大量的句子对，因为在许多情况下，只有几千个标记的数据点可用。幸运的是，有一种方法可以增加数据，这样当只有少量标记数据可用时，可以对嵌入模型进行微调。这一程序被称为 Augment SBET(增强 SBERT)。

1. 使用少量数据样本（gold dataset）微调 cross-encoder（BERT）。
2. 创建新的句子对。在其他具体领域中，可以收集该领域的文本数据。
3. 用微调后的 cross-encoder 模型为新的句子对打标签，标注后的句子对为 silver dataset。
4. 用增强后的数据集（gold + silver dataset）训练 SBERT 模型。

1. Fine-tune a cross-encoder (BERT) using a small, annotated dataset (gold dataset).
2. Create new sentence pairs.
3. Label new sentence pairs with the fine-tuned cross-encoder (silver dataset).
4. Train a bi-encoder (SBERT) on the extended dataset (gold + silver dataset).

Here, a gold dataset is a small but fully annotated dataset that holds the ground truth. A silver dataset is also fully annotated but is not necessarily the ground truth as it was generated through predictions of the cross-encoder.

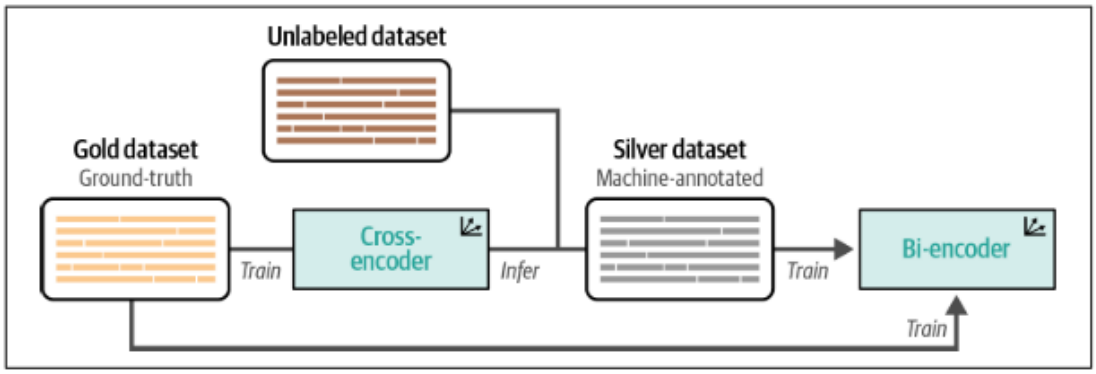


Figure 10-12. Augmented SBERT works through training a cross-encoder on a small gold dataset, then using that to label an unlabeled dataset to generate a larger silver dataset. Finally, both the gold and silver datasets are used to train the bi-encoder.

Cross-encoder、Bi-encoder 和 SBERT 的区别：

Cross-encoder 是一种深度学习模型架构，专门用来处理成对句子输入，并直接输出这对输入的相关性得分。它的工作原理是同时将两个句子输入到一个共享的编码器（通常是基于 Transformer 的模型，比如 BERT）中，这个编码器会考虑两个句子之间的交互信息，从而生成一个综合表示来预测它们的关系。由于 cross-encoder 能够捕捉句子间的复杂关系，因此通常在需要精确匹配或比较的任务中表现优异，例如问答系统中的答案选择或信息检索中的精确匹配任务。然而，其**计算成本较高**，因为每次评估句子对时都需要运行整个模型。

Bi-Encoder 则包含两个独立的编码器，每个编码器负责处理输入对中的一个句子。这两个编码器可以是相同的模型实例（共享权重，如 SBERT），也可以是不同的。Bi-encoder 分别对两个句子进行编码，产生各自的向量表示，然后通过简单的相似度计算方法（如余弦相似度）来衡量这两个句子的相似度。这种方法的优点在于计算效率高，尤其是在大规模数据检索场景下，可以通过预先计算和索引句子向量来加速查询过程。SBERT 实际上就是一种实现 bi-encoder 架构的具体方法，特别适用于句子级别的相似度搜索和语义匹配任务。

SBERT（Sentence-BERT）是 Bi-encoder 架构的一种具体实现，专为获得更有效的句子嵌入而设计。通过修改原始 BERT 模型的训练目标，SBERT 能够在保持句子语义信息的同时大幅减少计算时间，使其非常适合于大规模语料库的句子相似度比较和语义搜索任务。SBERT 的核心思想是在预训练的 BERT 模型基础上，通过特定任务（如自然语言推理或句子相似度任务）进一步微调，以优化句子级别的嵌入表示。

Augmented SBERT 通过 Cross-Encoder（教师模型）挖掘无标注数据中的知识，转化为 SBERT（学生模型）的训练信号，实现“以质量换数量”的数据增强。

Step 1: Fine-tune a cross-encoder

```python
import pandas as pd
from tqdm import tqdm
from datasets import load_dataset, Dataset
from sentence_transformers import InputExample
from sentence_transformers.datasets import NoDuplicatesDataLoader

# Prepare a small set of 10000 documents for the cross-encoder
dataset = load_dataset("glue", "mnli", split="train").select(range(10_000))
mapping = {2: 0, 1: 0, 0:1} #中性和矛盾标注0，蕴含标注1

# Data Loader
gold_examples = [
        InputExample(texts=[row["premise"], row["hypothesis"]], label=mapping[row["label"]])
        for row in tqdm(dataset)
]
gold_dataloader = NoDuplicatesDataLoader(gold_examples, batch_size=32)

# Pandas DataFrame for easier data handling
gold = pd.DataFrame(
        {
        'sentence1': dataset['premise'],
        'sentence2': dataset['hypothesis'],
        'label': [mapping[label] for label in dataset['label']]
        }
)
```

```
/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/s
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
100%|██████████| 10000/10000 [00:00<00:00, 28880.88it/s]
```

```python
from sentence_transformers.cross_encoder import CrossEncoder

# Train a cross-encoder on the gold dataset
cross_encoder = CrossEncoder('bert-base-uncased', num_labels=2)
cross_encoder.fit(
        train_dataloader=gold_dataloader,
        epochs=1,
        show_progress_bar=True,
        warmup_steps=100,
        use_amp=False
)
```

```
config.json: 100% 570/570 [00:00<00
model.safetensors: 100% 440M/440M
Some weights of BertForSequenceClassification were not initialized from the mo
You should probably TRAIN this model on a down-stream task to be able to use
tokenizer_config.json: 100% 48.0/48.
vocab.txt: 100% 232k/232k [00:00<00
tokenizer.json: 100% 466k/466k [00:
README.md: 100% 10.5k/10.5k [00:0
wandb: Currently logged in as: 1449185730 (1449185730-sun-yat-sen-university)
Tracking run with wandb version 0.21.0
Run data is saved locally in /content/wandb/run-20250811_134955-1vz12sdv
Syncing run vital-galaxy-2 to Weights & Biases (docs)
View project at https://wandb.ai/1449185730-sun-yat-sen-university/sentence-trans
View run at https://wandb.ai/1449185730-sun-yat-sen-university/sentence-transfor
[312/312 02:21, Epoch 1/1]
```

Step 2: Create new sentence pairs

```python
# Prepare the silver dataset by predicting labels with the cross-encoder
silver = load_dataset("glue", "mnli", split="train").select(range(10_000, 50_000))
pairs = list(zip(silver['premise'], silver['hypothesis']))
```

Step 3: Label new sentence pairs with the fine-tuned cross-encoder (silver dataset)

```
import numpy as np

# Label the sentence pairs using our fine-tuned cross-encoder
output = cross_encoder.predict(pairs, apply_softmax=True, show_progress_bar=True)
silver = pd.DataFrame(
        {
                "sentence1": silver["premise"],
                "sentence2": silver["hypothesis"],
                "label": np.argmax(output, axis=1)
        }
)
```

```
Batches: 100% 1250/1250 [03:07<00:00, 7.61it/s]
```

Step 4: Train a bi-encoder (SBERT) on the extended dataset (gold + silver dataset)

```
[7] # Combine gold + silver
data = pd.concat([gold, silver], ignore_index=True, axis=0)
data = data.drop_duplicates(subset=['sentence1', 'sentence2'], keep="first")
train_dataset = Dataset.from_pandas(data, preserve_index=False)
```

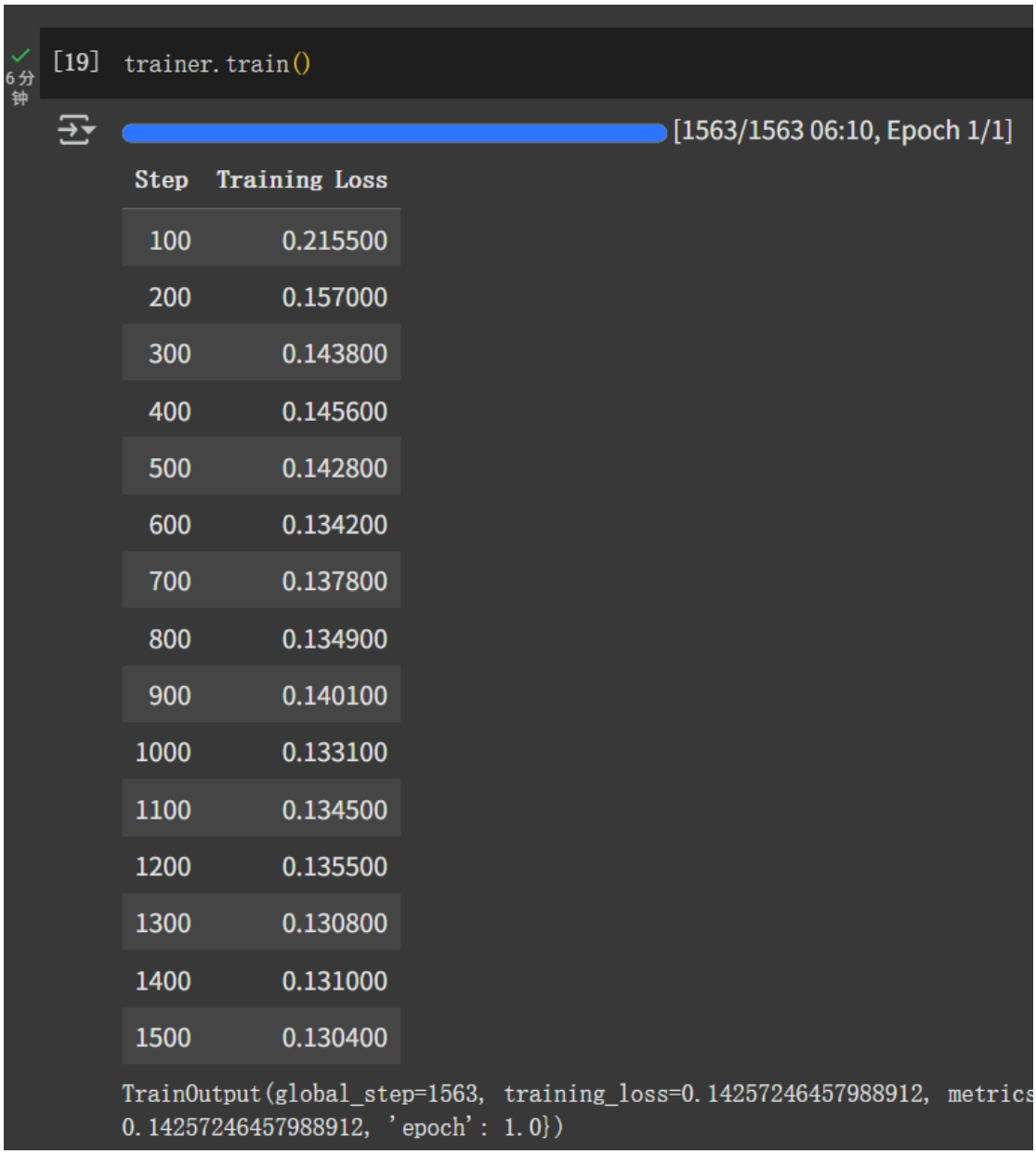
```
data
```

	sentence1	sentence2	label
0	Conceptually cream skimming has two basic dime...	Product and geography are what make cream skim...	0
1	you know during the season and i guess at at y...	You lose the things to the following level if ...	1
2	One of our number will carry out your instruct...	A member of my team will execute your orders w...	1
3	How do you know? All this is their information...	This information belongs to them.	1
4	yeah i tell you what though if you go price so...	The tennis shoes have a range of prices.	0
...	...	...	...
49995	It was a cop, a woman of intermediate age.	The police were after us.	0
49996	In Trinidad, Motel Las Cuevas has a disco with...	There is a disco in Motel Las Cuevas.	1
49997	Tommy's heart sank at the sight of them.	The sight of them cheered Tommy up.	0
49998	Wodehouse, Paul Gigot and Mary McGrory.	Paul Gigot and others.	1
49999	Democrats think they're immune to this attack ...	Democrats think they have covered both ends of...	1

49998 rows × 3 columns

```
from sentence_transformers.evaluation import EmbeddingSimilarityEvaluator

# Create an embedding similarity evaluator for stsb
val_sts = load_dataset('glue', 'stsb', split='validation')
evaluator = EmbeddingSimilarityEvaluator(
        sentences1=val_sts["sentence1"],
        sentences2=val_sts["sentence2"],
        scores=[score/5 for score in val_sts["label"]],
        main_similarity="cosine"
)
```

```python
from sentence_transformers import losses, SentenceTransformer
from sentence_transformers.trainer import SentenceTransformerTrainer
from sentence_transformers.training_args import SentenceTransformerTrainingArguments

# Define model
embedding_model = SentenceTransformer('bert-base-uncased')

# Loss function
train_loss = losses.CosineSimilarityLoss(model=embedding_model)

# Define the training arguments
args = SentenceTransformerTrainingArguments(
        output_dir="augmented_embedding_model",
        num_train_epochs=1,
        per_device_train_batch_size=32,
        per_device_eval_batch_size=32,
        warmup_steps=100,
        fp16=True,
        eval_steps=100,
        logging_steps=100,
)

# Train model
trainer = SentenceTransformerTrainer(
        model=embedding_model,
        args=args,
        train_dataset=train_dataset,
        loss=train_loss,
        evaluator=evaluator
)
```



```
[19] trainer.train()
```

[1563/1563 06:10, Epoch 1/1]

Step	Training Loss
100	0.215500
200	0.157000
300	0.143800
400	0.145600
500	0.142800
600	0.134200
700	0.137800
800	0.134900
900	0.140100
1000	0.133100
1100	0.134500
1200	0.135500
1300	0.130800
1400	0.131000
1500	0.130400

```
TrainOutput(global_step=1563, training_loss=0.14257246457988912, metrics
0.14257246457988912, 'epoch': 1.0})
```

```
[20] # Evaluate our trained model
evaluator(embedding_model)
```

```
{'pearson_cosine': 0.7083285360443494, 'spearman_cosine': 0.7241080175082656}
```

去掉 silver 数据集，保留 gold dataset 重新训练模型：

Step 5: Evaluate without silver dataset

```
# Only gold
data = pd.concat([gold], ignore_index=True, axis=0)
data = data.drop_duplicates(subset=['sentence1', 'sentence2'], keep="first")
train_dataset = Dataset.from_pandas(data, preserve_index=False)

# Define model
embedding_model = SentenceTransformer('bert-base-uncased')

# Loss function
train_loss = losses.CosineSimilarityLoss(model=embedding_model)

# Define the training arguments
args = SentenceTransformerTrainingArguments(
        output_dir="gold_only_embedding_model",
        num_train_epochs=1,
        per_device_train_batch_size=32,
        per_device_eval_batch_size=32,
        warmup_steps=100,
        fp16=True,
        eval_steps=100,
        logging_steps=100,
)

# Train model
trainer = SentenceTransformerTrainer(
        model=embedding_model,
        args=args,
        train_dataset=train_dataset,
        loss=train_loss,
        evaluator=evaluator
)
```

```
trainer.train()
```

```
WARNING:sentence_transformers.SentenceTransformer:No sentence-transformers model found with nam
[313/313 01:51, Epoch 1/1]
```

Step	Training Loss
100	0.226800
200	0.171400
300	0.160000

```
TrainOutput(global_step=313, training_loss=0.18524525797786043, metrics={'train_runtime': 111.3
0.18524525797786043, 'epoch': 1.0})
```

```
# Evaluate our trained model
evaluator(embedding_model)
```

```
{'pearson_cosine': 0.6209085190164768, 'spearman_cosine': 0.6475908454858665}
```

比起使用ombine gold and silver datasets的模型，只使用gold datasets的模型表现明显不如前者。

2.Unsupervised Learning

①TBSDA（基于 Transformer 的序列去噪自编码器）

TSDAE 用于创建无监督学习的嵌入模型，是一种利用 Transformer 架构的强大序列建模能力，以实现去噪自编码目标的神经网络模型。

TSDAE 的基本思想是，我们通过从输入句子中去除一定比例的单词来达到添加噪声的目的。这个“受损”的句子通过编码器和池化层，映射到一个句子嵌入。对于这个句子嵌入，解码器试图从“受损”句子中重构去噪的原始句子。

编码器和解码器共同组成了自编码器。自编码器是一种无监督学习模型，其目标是学习数据的高效表示（编码）。

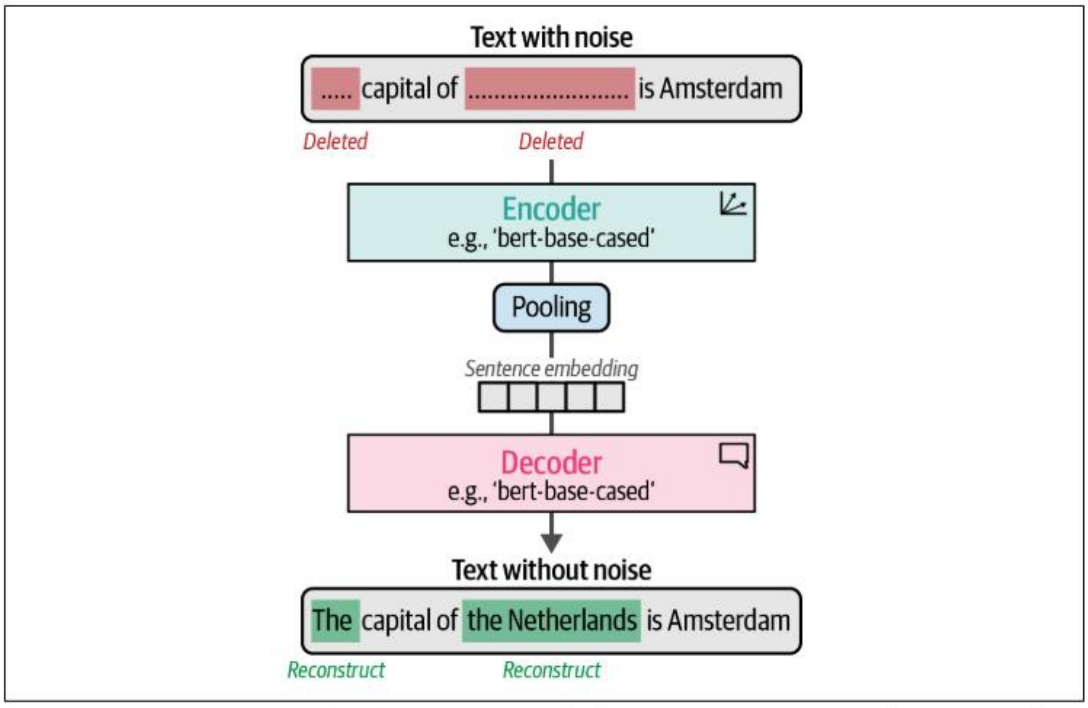


Figure 10-13. TSDAE randomly removes words from an input sentence that is passed through an encoder to generate a sentence embedding. From this sentence embedding, the original sentence is reconstructed.

这种方法非常类似于掩码语言建模，我们试图重建和学习某些掩码单词。在这里，我们尝试重建整个句子，而不是重建屏蔽词。BERT 本质上是 Transformer-Based Sequential Denoising Auto-Encoder 的一种，具体是 Masked Language Model。训练后，我们可以使用编码器从文本中生成嵌入，因为解码器只用于判断嵌入是否可以准确地重建原始句子。

由于我们只需要一堆没有任何标签的句子，因此训练这个模型很简单。我们首先

下载一个外部 tokenizer，用于去噪过程：

Unsupervised Learning

Tranformer-based Denoising AutoEncoder (TSDAE)

```
[9]  # Download additional tokenizer
     import nltk
     nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
True
```

这段代码用于下载 NLTK（自然语言工具包）中的 punkt 分词器资源，它提供文本处理功能（分词、词性标注、句法分析等）。例如将文本分割成单词（word tokenization）、将段落分割成句子（sentence tokenization）。

```
[10] from tqdm import tqdm
     from datasets import Dataset, load_dataset
     from sentence_transformers.datasets import DenoisingAutoEncoderDataset

     # Create a flat list of sentences; 加载GLUE数据集中的MNLI子集（自然语言推理）
     mnli = load_dataset("glue", "mnli", split="train").select(range(25_000))
     # 合并所有前提(premise)和假设(hypothesis)句子为一个平面列表
     flat_sentences = mnli["premise"] + mnli["hypothesis"]

     # Add noise to our input data
     damaged_data = DenoisingAutoEncoderDataset(list(set(flat_sentences)))#可以进一步在函数中定义删除单词的比例

     # Create dataset
     train_dataset = {"damaged_sentence": [], "original_sentence": []}
     for data in tqdm(damaged_data):
         train_dataset["damaged_sentence"].append(data.texts[0])  #损坏版本
         train_dataset["original_sentence"].append(data.texts[1])  #原始版本

     # 转换为Hugging Face Dataset格式
     train_dataset = Dataset.from_dict(train_dataset)
```

```
100%|██████████| 48353/48353 [00:16<00:00, 2990.89it/s]
```

```
train_dataset[0]
```

```
{'damaged_sentence': 'Danvers the to',
 'original_sentence': 'Danvers burned the draft treaty to destroy it.'}
```

```
[12] from sentence_transformers.evaluation import EmbeddingSimilarityEvaluator

     # Create an embedding similarity evaluator for stsb
     val_sts = load_dataset('glue', 'stsb', split='validation')
     evaluator = EmbeddingSimilarityEvaluator(
             sentences1=val_sts["sentence1"],
             sentences2=val_sts["sentence2"],
             scores=[score/5 for score in val_sts["label"]],
             main_similarity="cosine"
     )
```

```
[13] from sentence_transformers import models, SentenceTransformer

     # Create your embedding model
     word_embedding_model = models.Transformer('bert-base-uncased')#创建词嵌入模型
     pooling_model = models.Pooling(word_embedding_model.get_word_embedding_dimension(), 'cls')#创建池化层模型
     #将上面两个模型拼接
     embedding_model = SentenceTransformer(modules=[word_embedding_model, pooling_model])
```

创建词嵌入模型时使用 models.Transformer 类初始化一个基于 Transformer 的词嵌入模型。该模型会将输入文本转换成词级别的嵌入向量（每个词对应一个向量）。输出维度[seq_len, 768]

创建池化层模型时使用 models.Pooling 将词级别的嵌入向量聚合成一个固定长度的句子嵌入向量。第一个参数用于获取词嵌入的维度。第二个参数‘cls’指定池化方法。这里使用 CLS 令牌的向量作为整个句子的表示。其他选项包括’mean’（均值池化）、’max’（最大池化）等。在预训练期间，不断更新 [CLS] 对应的向量来捕获整个序列的语义信息，即作为整个句子的表示。BERT 训练时通常使用该标记的向量进行分类任务。输出维度[1, 768]。

```
from sentence_transformers import losses

# Use the denoising auto-encoder loss
train_loss = losses.DenoisingAutoEncoderLoss(
        embedding_model, tie_encoder_decoder=True
)
train_loss.decoder = train_loss.decoder.to("cuda")
```

```
from sentence_transformers.trainer import SentenceTransformerTrainer
from sentence_transformers.training_args import SentenceTransformerTrainingArguments

# Define the training arguments
args = SentenceTransformerTrainingArguments(
        output_dir="tsdae_embedding_model",
        num_train_epochs=1,
        per_device_train_batch_size=16,
        per_device_eval_batch_size=16,
        warmup_steps=100,
        fp16=True,
        eval_steps=100,
        logging_steps=100,
)

# Train model
trainer = SentenceTransformerTrainer(
        model=embedding_model,
        args=args,
        train_dataset=train_dataset,
        loss=train_loss,
        evaluator=evaluator
)
trainer.train()
```

```
[16] # Evaluate our trained model
     evaluator(embedding_model)

     {'pearson_cosine': 0.7468797415583491, 'spearman_cosine': 0.7515621538923806}
```

②使用 TBSDA 进行领域自适应

虽然无监督学习在很少或没有可用的标记数据时可以发挥作用，但是无监督学习的训练效果不及监督学习，并且难以学习特定领域的概念。这时 Domain adaptation（领域自适应）便有了用武之地。Domain adaptation 的目标是将现有的嵌入模型从原有的领域更新到包含不同主题的特定文本领域。图 10-14 展示了域在内容上的差异。Target domain or out-domain 目标域或域外通常包含了 source domain or in-domain 源域或域内找不到的单词和主题。

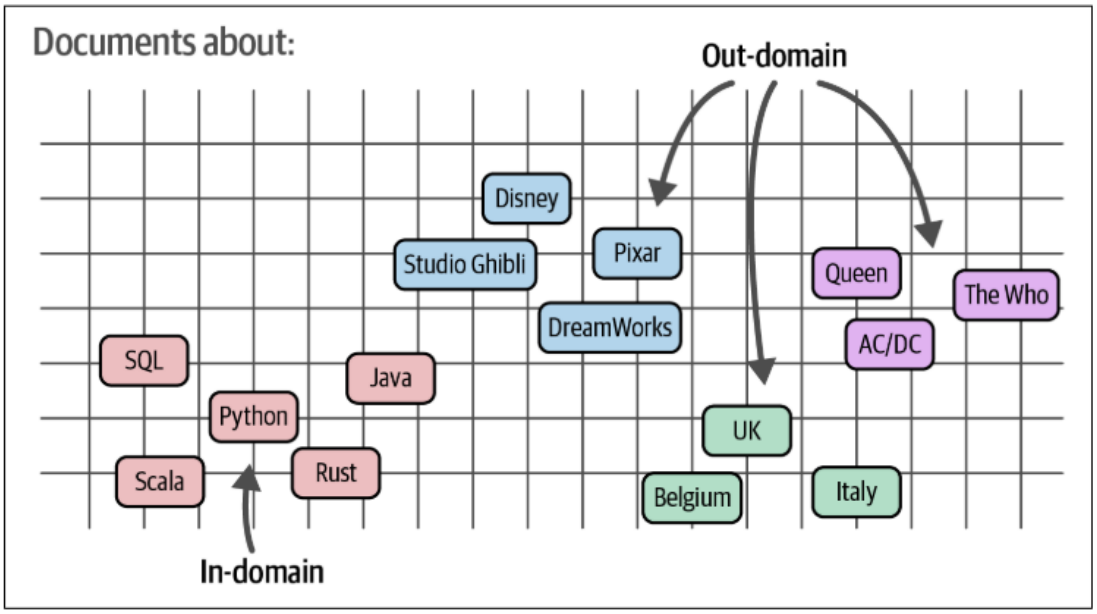


Figure 10-14. In domain adaptation, the aim is to create and generalize an embedding model from one domain to another.

一种用于领域自适应的方法被称为 **adaptive pretraining** 自适应预训练。首先，使用无监督技术对特定领域的语料库进行预训练，例如前面讨论的 TSDAE 或掩码语言建模。如图 10-15 所示，可以使用 out-domain 或 target domain 中的训练数据集来微调该模型。虽然来自 target domain 的数据是首选，但 out-domain 的数据也可以工作，因为我们一开始就是在 target domain 上进行无监督训练。

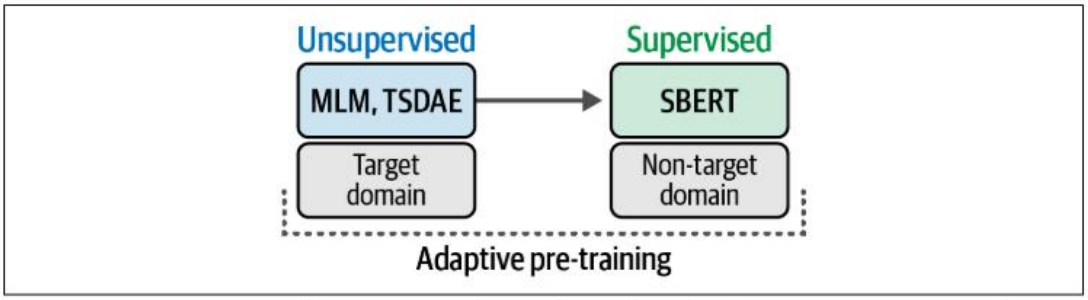


Figure 10-15. Domain adaptation can be performed with adaptive pretraining and adaptive fine-tuning.